

INFORMATION SYSTEMS AND DATABASES – PORTFOLIO ELEMENT 3

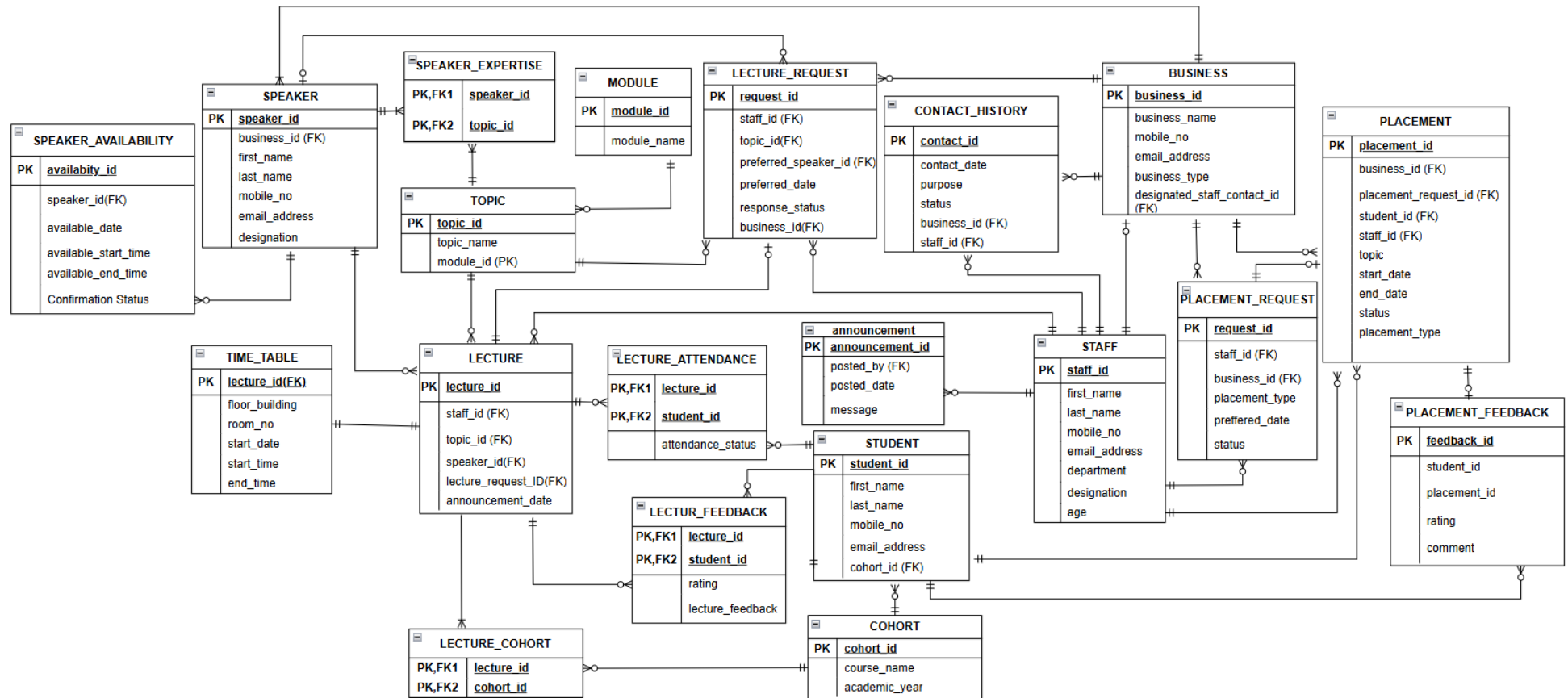


Student Name: Abhey Saini

1)ASSUMPTIONS/BUSINESS RULES:

- Each lecture is assigned to only one topic but a topic could have one or more lectures.
- A lecture will be arranged by only one staff member.
- There is only one assigned speaker for a particular lecture but the lecture may be for a cohort or more than one cohort.
- Only students can give feedback to the guest lectures. Only one feedback per student to a particular lecture moreover a student can give feedback to multiple lectures. However, submitting feedback is optional.
- It is optional for a lecture to get feedback, but if a lecture gets feedback there can be one or more feedback to a particular lecture by different student.
- Each lecture will have a unique timetable.
- Each speaker is connected to only one business, but a business can have one or more than one speaker. It is must for business to have at least one speaker.
- A speaker can have one or more expertise.
- It is not mandatory for speaker to have available slots for guest lectures but if a speaker is available then, he/ she can have more one or more than one slots for lectures
- It is optional for a business to provide placement, but the placement should belong to only one business. Moreover, a business can provide one or more than one placement.
- It is optional for placement to have feedback but only there will one feedback for each placement. A student can give feedback to multiple placements.
- A student may or may not go for placement, but if they do, they can have one or more placements.
- A student can belong to only one cohort and for cohort may or not have students.
- only one staff member is designated to a business. staff may or not connected to business.
- It is optional for staff to make lecture request, placement request and make contact to business if they do, they can request or contact multiple times.
- A request for lecture is dedicated to only single topic.
- It is optional for a topic to be requested for lecture. However, a topic can be requested one or more than one times
- There is a contact history table to track the communication between staff members and businesses. It is optional that a business is contacted if there is communication, a business can have one or more than one communication history.
- There should be announcement table to track all the indispensable announcements regarding placements, guest lectures, and so on

2) ER DIAGRAM:



3) ENTITY SPECIFICATION FORMS

ENTITY NAME: STAFF Entity description: stores details of all the staff members of the university				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
Staff_id	NUMBER (4)	pk		101
First_name	VARCHAR2(30)	nn		anuradha
Last_name	VARCHAR2(25)	nn		sethi
mobile_no	VARCHAR2(15)	nn		+449876543567
email_address	VARCHAR2(254)	nn		anuradhaofficial@gmail.com
department	VARCHAR2(10)	nn		computing
designation	VARCHAR2 (20)	nn		professor
age	NUMBER(2)	nn	Age should be between 20 to 70	30

ENTITY NAME: LECTURE Entity description: stores details of all the lectures arranged by staff members				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
lecture_id	NUMBER (5)	pk		1001
staff_id	NUMBER (4)	fk		101
topic_id	VARCHAR2 (8)	fk		T1000001
speaker_id	NUMBER (4)	fk		2001
Lecture_request_id	VARCHAR2 (5)	fk		R1001
announcement_date	DATE			23 Jan 2025

ENTITY NAME: SPEAKER Entity description: stores details of all the external speakers				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
speaker_id	NUMBER (4)	pk		2001
first_name	VARCHAR2(30)	nn		aman
last_name	VARCHAR2(25)	nn		khurana
email_address	VARCHAR2(254)	nn		khurana@gmail.com
mobile_no	VARCHAR2(15)	nn		+449087658907
designation	VARCHAR2 (15)	nn		ai expert
business_id	VARCHAR2 (5)	fk		BS101

ENTITY NAME: SPEAKER_EXPERTISE Entity description: stores details of speaker expertise				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
speaker_id	NUMBER (4)	pk, fk		2001
topic_id	VARCHAR2 (8)	pk, fk		T1000001

ENTITY NAME: BUSINESS Entity description: stores details of local business and staff members connected with it.				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
business_id	VARCHAR2 (5)	pk		BS101
business_name	VARCHAR2 (100)	nn		ai heath care
email_address	VARCHAR2 (254)	nn		officialai@gmail.com
mobile_no	VARCHAR2 (15)	nn		+449086112345
business_type	VARCHAR2 (30)	nn		health care tools
designated_staff_contact_id	NUMBER (4)	fk		101

ENTITY NAME: PLACEMENT Entity description: stores details placements provided by different businesses				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
Placement_id	VARCHAR2 (5)	pk		101
placement_request_id	VARCHAR2 (5)	fk		PR101
business_id	VARCHAR2 (5)	fk		BS101
Student_id	NUMBER (6)	fk		100001
Staff_id	NUMBER (4)	fk		101
topic	VARCHAR2 (50)	nn		data communication
Placement_type	VARCHAR2 (25)	nn		internship
start_date	DATE	nn		1 Jan 2025
end_date	DATE	nn		4 April 2025
status	VARCHAR2(10)		It should be either pending, placed or not placed	placed

ENTITY NAME: PLACEMENT_REQUEST Entity description: stores details of all the requests sent for placement to businesses				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
request_id	VARCHAR2 (5)	pk		PR101
staff_id	NUMBER (4)	fk		101
business_id	VARCHAR2 (5)	fk		BS101
placement_type	VARCHAR2 (15)	nn		internship
preferred_date	DATE	nn		23 April 2025
status	VARCHAR2 (15)	nn	Either it is pending, approved or disapproved	approved

ENTITY NAME: <u>PLACEMENT_FEEDBACK</u> Entity description: stores details of the feedbacks given by the students to the different placements				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
feedback_id	VARCHAR2 (6)	pk		PD1001
student_id	NUMBER (6)	fk		100001
placement_id	VARCHAR2 (5)	fk		101
rating	VARCHAR2 (5)	nn	It could be 1,2,3,4 or 5	5
comment	VARCHAR2 (250)	nn		Fantastic experience

ENTITY NAME: <u>STUDENT</u> Entity description: stores details of students				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
student_id	NUMBER (6)	pk		100001
first_name	VARCHAR2(30)	nn		rishab
last_name	VARCHAR2(25)	nn		khanna
email_address	VARCHAR2(254)	nn		khanna@gmail.com
mobile_no	VARCHAR2(15)	nn		+440987654321
Cohort_id	VARCHAR2 (6)	fk		CO1001

ENTITY NAME: COHORT Entity description: stores details of cohort				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
cohort_id	VARCHAR2 (6)	pk		CO1001
course_name	VARCHAR2 (50)	nn		computer science
adademic_year	DATE	nn		2025

ENTITY NAME: LECTURE_COHORT Entity description: stores details of which cohorts attended the guest lectures				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
Lecture_id	NUMBER (5)	pk		1001
Cohort_id	VARCHAR2 (6)	pk		CO1001

ENTITY NAME: TOPIC Entity description: stores details of topics				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
topic_id	VARCHAR2 (8)	pk		T1000001
module_id	VARCHAR2(5)	nn		ML101
topic_name		nn		algorithms

ENTITY NAME: MODULE Entity description: Store details of all the modules offered by the university				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
module_id	VARCHAR2(5)	nn		ML101
Module_name		nn		Information systems and databases

ENTITY NAME: CONTACT_HISTORY Entity description: stores details of all the contact history made by staff members to businesses				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
contact_id	VARCHAR2 (7)	pk		CON1001
Contact_date	DATE	nn		25 Feb 2025
purpose	VARCHAR2(25)	nn		guest lecture
status	VARCHAR2(8)	nn	It is either accepted or, pending declined	accepted
business_id	VARCHAR2 (5)	fk		BS101
staff_id	NUMBER (4)	fk		101

ENTITY NAME: LECTURE_REQUEST Entity description: stores details of all the requests made by the staff for guest lectures				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
request_id	VARCHAR2 (5)	pk		R1001
staff_id	NUMBER (4)	fk		101
topic_id	VARCHAR2 (8)	fk		T1000001
Preferred date	DATE	nn		1 Jan 2025
preferred speaker_id	NUMBER (4)	pk		2001
response_status	VARCHAR2(8)	nn	It is either accepted or, pending declined	accepted
business_id	VARCHAR2 (5)	fk		BS101

ENTITY NAME: TIME_TABLE Entity description: stores details of when lecture will be conducted, where and so on				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
lecture_id	NUMBER (5)	fk		1001
floor_building	VARCHAR (20)	nn		3 pigeon
room_no	NUMBER (4)	nn		101
start_date	DATE	nn		23 Jan 2025
start_time	DATE	nn		9.30 am
end_time	DATE	nn		11.30 am

ENTITY NAME: LECTURE_ATTENDANCE Entity description: stores details of which students attended the lectures				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
lecture_id	NUMBER (5)	fk		1001
student_id	NUMBER (6)	fk		100001
Attendance_status	VARCHAR2 (7)	nn	It should be either present, absent or late	present

ENTITY NAME: LECTURE_FEEDBACK Entity description: stores details of all the feedback given by students				
Attributes	Data types and width	Status	Validation*	Example of input and any relevant information
Lecture_id	NUMBER (5)	Pk, fk		1001
Student_id	NUMBER (6)	Pk, fk		100001
rating	NUMBER (5)	nn	It should be between 1 and 5	5
Lecture_feedback	VARCHAR2 (250)			excellent

ENTITY NAME: SPEAKER AVAIBILITY Entity description: stores details of the speaker avaibility				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
Available_id	VARCHAR2 (7)	pk		AVL1001
Speaker_id	NUMBER (4)	fk		2001
Available_date	DATE	nn		10 May 2025
Available_start_time	DATE	nn		09:00:00
Available_end_time	DATE	nn		01.00.00
Confirmation_Status	VARCHAR2(3)	nn	It could be either no or yes	yes

ENTITY NAME: ANNOUNCEMENT Entity description: stores details of announcements made by staff members.				
Attributes	Data types and width	Status*	Validation	Example of input and any relevant information
Announcement_id	NUMBER (5)	fk		1001
posted_by	NUMBER (4)	fk		101
Posted_date	DATE	nn		12 Jan 2025
message	VARCHAR2(300)			

SQL TABLE CREATION SCRIPTS

```

CREATE TABLE lecture (
lecture_id          NUMBER (5) ,
staff_id            NUMBER (4) NOT NULL,
topic_id            VARCHAR2 (8) NOT NULL,
speaker_id          NUMBER(4) NOT NULL,
lecture_request_id  VARCHAR2(5) NOT NULL,
announcement_date   DATE NOT NULL,
CONSTRAINT pk_lecture_lecture_id PRIMARY KEY (lecture_id),
CONSTRAINT fk_lecture_staff_id FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
CONSTRAINT fk_lecture_topic_id FOREIGN KEY (topic_id) REFERENCES topic(topic_id),
CONSTRAINT fk_lecture_speaker_id FOREIGN KEY (speaker_id) REFERENCES speaker(speaker_id),
CONSTRAINT fk_lecture_lecture_request_id FOREIGN KEY (lecture_request_id) REFERENCES lecture_request(Request_id)
);

```

```

CREATE TABLE speaker (
speaker_id          NUMBER(4),
first_name          VARCHAR2 (30) NOT NULL,
last_name           VARCHAR2 (25) NOT NULL,
email_address       VARCHAR2 (254) NOT NULL,
mobile_no           VARCHAR2 (15) NOT NULL,
designation          VARCHAR2 (15) NOT NULL,
business_id         VARCHAR2 (5) NOT NULL,
CONSTRAINT pk_speaker_speaker_id PRIMARY KEY (speaker_id) ,
CONSTRAINT fk_speaker_business_id FOREIGN KEY (business_id) REFERENCES business(business_id)
);

```

```

CREATE TABLE staff (
staff_id            NUMBER(4),
first_name          VARCHAR2(30) NOT NULL,
last_name           VARCHAR2(25) NOT NULL,
mobile_no           VARCHAR2(15) NOT NULL,
email_address       VARCHAR2(254) NOT NULL,
department           VARCHAR2(10) NOT NULL,
designation          VARCHAR2(20) NOT NULL,
age                 NUMBER(2) NOT NULL,
CONSTRAINT pk_staff_staff_id PRIMARY KEY (staff_id),
CONSTRAINT ck_staff_age CHECK (age BETWEEN 20 and 70)
);

```

```

CREATE TABLE speaker_expertise (
Speaker_id          NUMBER (4),
Topic_id            VARCHAR2 (8),
CONSTRAINT pk_speaker_expertise PRIMARY KEY (speaker_id,topic_id),
CONSTRAINT fk_speaker_expertise_speaker_id FOREIGN KEY (speaker_id) REFERENCES speaker(speaker_id),
CONSTRAINT fk_speaker_expertise_topic_id FOREIGN KEY (topic_id) REFERENCES topic(topic_id)
);

```

```

CREATE TABLE placement (
Placement_id          VARCHAR2 (5),
Placement_request_id  VARCHAR2 (5) ,
Business_id           VARCHAR2 (5) NOT NULL,
Student_id            NUMBER (6) NOT NULL,-
Staff_id              NUMBER (4) NOT NULL,
Topic                 VARCHAR2 (50) NOT NULL,
Placement_type        VARCHAR2 (25) NOT NULL,
Start_date            DATE NOT NULL,
End_date              DATE NOT NULL,
Status                VARCHAR2 (10) NOT NULL,
CONSTRAINT pk_placement_placement_id PRIMARY KEY (placement_id),
CONSTRAINT fk_placement_placement_request_id FOREIGN KEY (placement_request_id) REFERENCES placement_request(request_id),
CONSTRAINT fk_placement_business_id FOREIGN KEY (business_id) REFERENCES business(business_id),
CONSTRAINT fk_placement_student_id FOREIGN KEY (student_id) REFERENCES student(student_id),
CONSTRAINT fk_placement_staff_id FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
CONSTRAINT ck_placement_status CHECK (status in ('placed', 'pending', 'not placed'))
);

```

```

CREATE TABLE business (
business_id           VARCHAR2(5) ,
business_name         VARCHAR2 (100) NOT NULL,
email_address         VARCHAR2 (254) NOT NULL,
mobile_no             VARCHAR2 (15) NOT NULL,
business_type         VARCHAR2 (30) NOT NULL,
designated_staff_contact_id  NUMBER (4) NOT NULL,
CONSTRAINT pk_business_business_id PRIMARY KEY (business_id),
CONSTRAINT fk_business_designated_staff_contact_id FOREIGN KEY (designated_staff_contact_id) REFERENCES staff(staff_id)
);

```

```

CREATE TABLE placement_request(
Request_id      VARCHAR2 (5),
staff_id        NUMBER (4)    NOT NULL,
business_id     VARCHAR2 (5)  NOT NULL,
placement_type  VARCHAR2 (15) NOT NULL,
preferred_date  DATE NOT NULL,
status          VARCHAR2 (15) NOT NULL,
CONSTRAINT pk_placement_request_request_id PRIMARY KEY (request_id),
CONSTRAINT fk_placement_request_staff_id FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
CONSTRAINT fk_placement_request_business_id FOREIGN KEY (business_id) REFERENCES business(business_id),
CONSTRAINT ck_placement_request_status CHECK (status in ('pending', 'approved', 'disapproved'))

);

```

```

CREATE TABLE placement_feedback (
Feedback_id     VARCHAR2 (6) ,
Student_id      NUMBER (6)    NOT NULL,
Placement_id    VARCHAR2 (5)  NOT NULL,
Rating          VARCHAR2 (5)  NOT NULL,
Comments        VARCHAR2 (250) NOT NULL,
CONSTRAINT pk_placement_feedback_feedback_id PRIMARY KEY (feedback_id) ,
CONSTRAINT fk_placement_feedback_student_id FOREIGN KEY (student_id) REFERENCES student(student_id),
CONSTRAINT fk_placement_feedback_placement_id FOREIGN KEY (placement_id) REFERENCES placement(placement_id),
CONSTRAINT ck_placement_feedback_rating CHECK (rating in(1,2,3,4,5))

);

```

```

CREATE TABLE cohort (
Cohort_id       VARCHAR2 (6),
Course_name     VARCHAR2 (50) NOT NULL,
Academic_year   DATE    NOT NULL,
CONSTRAINT pk_cohort_cohort_id PRIMARY KEY (cohort_id)

);

```

```
CREATE TABLE student (  
  Student_id      NUMBER(6),  
  First_name      VARCHAR2 (30) NOT NULL,  
  Last_name       VARCHAR (25) NOT NULL,  
  Email_address   VARCHAR2 (254) NOT NULL,  
  Mobile_no       VARCHAR2 (15) NOT NULL,  
  Cohort_id       VARCHAR2 (6) NOT NULL,  
  CONSTRAINT pk_student_student_id PRIMARY KEY (student_id),  
  CONSTRAINT fk_student_cohort_id FOREIGN KEY (cohort_id) REFERENCES cohort(cohort_id)  
  
);
```

```
CREATE TABLE lecture_cohort (  
  Lecture_id      NUMBER (5),  
  Cohort_id       VARCHAR2 (6) ,  
  CONSTRAINT pk_lecture_cohort PRIMARY KEY (lecture_id, cohort_id)  
);
```

```
CREATE TABLE topic (  
  Topic_id       VARCHAR2 (8) ,  
  Module_id      VARCHAR2 (5) NOT NULL,  
  Topic_name     VARCHAR2 (50) NOT NULL,  
  CONSTRAINT pk_topic_topic_id PRIMARY KEY (topic_id),  
  CONSTRAINT fk_topic_module_id FOREIGN KEY (module_id) REFERENCES module(module_id)  
  
);
```

```
CREATE TABLE module (  
  Module_id      VARCHAR2 (5),  
  Module_name    VARCHAR2 (250) NOT NULL,  
  CONSTRAINT pk_module_module_id PRIMARY KEY (module_id)  
);
```

```

CREATE TABLE contact_history (
Contact_id          VARCHAR2 (7) ,
contact_Date        DATE NOT NULL,
Purpose             VARCHAR2 (25) NOT NULL,
Status              VARCHAR2 (8) NOT NULL,
Business_id         VARCHAR2 (5) NOT NULL,
Staff_id            NUMBER (4) NOT NULL,
CONSTRAINT pk_contact_history_contact_id PRIMARY KEY (contact_id),
CONSTRAINT fk_contact_history_business_id FOREIGN KEY (business_id) REFERENCES business(business_id),
CONSTRAINT fk_contact_history_staff_id FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
CONSTRAINT ck_contact_history_status CHECK (status in('pending', 'accepted','declined'))

);

```

```

CREATE TABLE time_table(
Lecture_id          NUMBER (5),
Floor_building      VARCHAR2 (20) NOT NULL,
Room_no             NUMBER (4) NOT NULL,
start_Date          DATE NOT NULL,
Start_time           DATE NOT NULL,
End_time            DATE NOT NULL,

CONSTRAINT pk_time_table_lecture_id PRIMARY KEY (lecture_id),
CONSTRAINT uq_time_slot UNIQUE ( start_date, start_time,end_time),
CONSTRAINT fk_time_table_lecture_id FOREIGN KEY (lecture_id) REFERENCES lecture(lecture_id)

);

```

```

CREATE TABLE lecture_request (
Request_id          VARCHAR2 (5),
Staff_id            NUMBER (4) NOT NULL,
Topic_id            VARCHAR2 (8) NOT NULL,
Preferred_date       DATE NOT NULL,
Preferred_speaker_id NUMBER (4) ,
Response_status      VARCHAR2 (8) NOT NULL,
Business_id         VARCHAR2 (5) NOT NULL,
CONSTRAINT pk_lecture_request_request_id PRIMARY KEY (request_id),
CONSTRAINT fk_lecture_request_staff_id FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
CONSTRAINT fk_lecture_request_topic_id FOREIGN KEY (topic_id) REFERENCES topic(topic_id),
CONSTRAINT fk_lecture_request_preferred_speaker_id FOREIGN KEY (preferred_speaker_id) REFERENCES speaker(speaker_id),
CONSTRAINT fk_lecture_request_business_id FOREIGN KEY (business_id) REFERENCES business(business_id),
CONSTRAINT ck_lectutre_request_response_status CHECK (response_status IN ('pending', 'accepted', 'declined'))

);

```



```

CREATE TABLE lecture_attendance(
lecture_id          NUMBER (5),
student_id          NUMBER(6),
attendance_status   VARCHAR2 ( 7) NOT NULL,
CONSTRAINT pk_lecture_attendance PRIMARY KEY (lecture_id, student_id),
CONSTRAINT ck_lecture_attendance_attendance_status CHECK (attendance_status in('present', 'absent', 'late'))

);

```

```

CREATE TABLE lecture_feedback(
Lecture_id          NUMBER (5),
Student_id          NUMBER (6),
Rating              NUMBER (5),
lecture_feedback     VARCHAR2 (250),
CONSTRAINT pk_lecture_feedback PRIMARY KEY (lecture_id, student_id),
CONSTRAINT fk_lecture_feedback_lecture_id FOREIGN KEY (lecture_id) REFERENCES LECTURE(LECTURE_ID),
CONSTRAINT fk_lecture_feedback_student_id FOREIGN KEY (student_id) REFERENCES student(student_ID),
CONSTRAINT ck_lecture_request_rating CHECK (rating in (1,2,3,4,5))

);

```

```

CREATE TABLE speaker_availability(
available_id         VARCHAR2(7),
speaker_id           NUMBER (4) NOT NULL,
available_date        DATE NOT NULL,
available_start_time  DATE NOT NULL,
available_end_time    DATE NOT NULL,
confirmation_status   VARCHAR2 (3) NOT NULL,
CONSTRAINT pk_speaker_availability_available_id PRIMARY KEY (available_id),
CONSTRAINT fk_speaker_availability_speaker_id FOREIGN KEY (speaker_id) REFERENCES speaker(speaker_id),
CONSTRAINT ck_speaker_availability_confirmation_status CHECK(confirmation_status in ('yes','no'))

);

```

```

CREATE TABLE announcement (
Announcement_id      NUMBER (5),
Posted_by            NUMBER(4) NOT NULL,
Posted_date           DATE NOT NULL,
Message              VARCHAR2 (300) NOT NULL,
CONSTRAINT pk_announcement_announcement_id PRIMARY KEY (announcement_id),
CONSTRAINT fk_announcement_posted_by FOREIGN KEY (posted_by) REFERENCES staff(staff_id)

);

```

SAMPLE DATA

INSERT ALL

```
INTO LECTURE VALUES (1001,101,'T1000001',2001,'R1001',TO_DATE('10 jan 2025', 'DD Month YYYY'))
INTO LECTURE VALUES (1002, 102,'T1000002',2002,'R1002',TO_DATE('10 jan 2025', 'DD Month YYYY'))
SELECT * FROM dual;
```

INSERT ALL

```
INTO SPEAKER VALUES (2001, 'aman','khurana','khurana@gmail.com','+449087658907','ai expert','BS101')
INTO SPEAKER VALUES (2002, 'kamla','sethi','kamla@gmail.com','+449087654321','ml expert','BS102')
SELECT * FROM dual;
```

INSERT ALL

```
INTO STAFF VALUES (101, 'anuradha','sethi','+449876543567','anuradhaofficial@gmail.com','computing','professor',30)
INTO STAFF VALUES (102, 'rama','kanna','+440987654323','rama@gmail.com','computing','professor',45)
SELECT * FROM dual;
```

INSERT ALL

```
INTO SPEAKER_EXPERTISE VALUES (2001, 'T1000001')
INTO SPEAKER_EXPERTISE VALUES (2001, 'T1000002')
SELECT * FROM dual;
```

INSERT ALL

```
INTO PLACEMENT VALUES (101,'PR101','BS101','100001',101,'data communication','internship',TO_DATE('1 Jan 2025', 'DD Month YYYY'),
TO_DATE('4 April 2025', 'DD Month YYYY'),'placed')
INTO PLACEMENT VALUES (102,'PR102','BS102',100002,102,'ai development','intership',TO_DATE('23 feb 2025', 'DD Month YYYY'),
TO_DATE('4 may 2025', 'DD Month YYYY'),'placed')
SELECT * FROM dual;
```

INSERT ALL

```
INTO BUSINESS VALUES ('BS101', 'ai heath care','officialai@gmail.com','+449086112345','health care tools',101)
INTO BUSINESS VALUES ('BS102', 'ai semesters','officailsemesters@gmail.com','+440987234567','ai applications',102)
SELECT * FROM dual;
```

```
INSERT ALL
| INTO PLACEMENT_REQUEST VALUES ('PR101',101,'BS101','internship',TO_DATE('23 April 2025', 'DD Month YYYY'),'approved')
| INTO PLACEMENT_REQUEST VALUES ('PR102',102,'BS102','project',TO_DATE('23 may 2025', 'DD Month YYYY'),'approved')
SELECT * FROM dual;
```

```
INSERT ALL
| INTO PLACEMENT_FEEDBACK VALUES ('PD1001',100001,101,5,'fantastic experience')
| INTO PLACEMENT_FEEDBACK VALUES('PD1002',100002,102,5,'good')
SELECT * FROM dual;
```

```
INSERT ALL
| INTO COHORT VALUES ('C01001', 'computer science',TO_DATE('05 February 2025', 'DD Month YYYY'))
| INTO COHORT VALUES ('C01002', 'cyber security',TO_DATE('07 February 2025', 'DD Month YYYY'))
SELECT * FROM dual;
```

```
INSERT ALL
| INTO STUDENT VALUES (100001,'rishab','khanna','khanna@gmail.com','+440987654321','C01001')
| INTO STUDENT VALUES (100002,'sanam','rana','rana@gmail.com','+448978675645','C01001')
SELECT * FROM dual;
```

```
INSERT ALL
| INTO LECTURE_COHORT VALUES (1001,'C01001')
| INTO LECTURE_COHORT VALUES (1001,'C01002')
SELECT * FROM dual;
```

```
INSERT ALL
| INTO TOPIC VALUES ('T1000001', 'ML102','algorithms')
| INTO TOPIC VALUES ('T1000002', 'ML101','relational database')
SELECT * FROM dual;
```

```
INSERT INTO MODULE VALUES('ML101','Information systems and databases');
INSERT INTO MODULE VALUES('ML102','machine learning');
```

```
INSERT ALL
  INTO CONTACT_HISTORY VALUES ('CON1001',TO_DATE('25 Feb 2025', 'DD Month YYYY'),'guest lecture','accepted','BS101',101)
  INTO CONTACT_HISTORY VALUES ('CON1002', TO_DATE('26 Feb 2025', 'DD Month YYYY'),'guest lecture','accepted','BS102',102)
SELECT * FROM dual;
```

```
INSERT ALL
  INTO TIME_TABLE VALUES (1001,'3 pigeon',101,TO_DATE('23 Jan 2025', 'DD Month YYYY'), TO_DATE('23 Jan 2025 09:30 AM', 'DD-MM-YYYY HH:MI AM'),
  TO_DATE('23 Jan 2025 11:30 AM', 'DD-MM-YYYY HH:MI AM'))
  INTO TIME_TABLE VALUES (1002,'3 pigeon',102,TO_DATE('24 Jan 2025', 'DD Month YYYY'),TO_DATE('24 Jan 2025 09:30 AM', 'DD-MM-YYYY HH:MI AM'),
  TO_DATE('24 Jan 2025 11:30 AM', 'DD-MM-YYYY HH:MI AM'))
SELECT * FROM dual;
```

```
INSERT ALL
  INTO LECTURE_REQUEST VALUES ('R1001',101,'T1000001',TO_DATE('1 jan 2025', 'DD Month YYYY'),2001,'accepted','BS101')
  INTO LECTURE_REQUEST VALUES ('R1002', 102,'T1000002',TO_DATE('5 jan 2025', 'DD Month YYYY'),2002,'accepted','BS102')
SELECT * FROM dual;
```

```
INSERT ALL
  INTO LECTURE VALUES (1001,101,'T1000001',2001,'R1001',TO_DATE('10 jan 2025', 'DD Month YYYY'))
  INTO LECTURE VALUES (1002, 102,'T1000002',2002,'R1002',TO_DATE('10 jan 2025', 'DD Month YYYY'))
SELECT * FROM dual;
```

```
INSERT ALL
  INTO LECTURE_FEEDBACK VALUES (1001,100001,5,'excellent')
  INTO LECTURE_FEEDBACK VALUES (1002,100002,4,'good')
SELECT * FROM dual;
```

```
INSERT ALL
  INTO SPEAKER_AVAILABILITY VALUES ('AVL1001',2001,TO_DATE('10 May 2025', 'DD Month YYYY'),TO_DATE('10 May 2025 09:00:00', 'DD-MM-YYYY HH24:MI:SS'),
  TO_DATE('10 May 2025 12:00:00', 'DD-MM-YYYY HH24:MI:SS'),'yes')
  INTO SPEAKER_AVAILABILITY VALUES('AVL1002',2002,TO_DATE('20 May 2025', 'DD Month YYYY'),TO_DATE('20 May 2025 09:00:00', 'DD-MM-YYYY HH24:MI:SS'),
  TO_DATE('20 May 2025 12:00:00', 'DD-MM-YYYY HH24:MI:SS'),'yes')
SELECT * FROM dual;
```

```

INSERT ALL
  INTO ANNOUNCEMENT VALUES (1001,101,TO_DATE('12 JAN 2025', 'DD Month YYYY'),'Guest lecture on algorithms scheduled for 23 January 2025 at 9:30 AM.')
  INTO ANNOUNCEMENT VALUES (1002,101,TO_DATE('19 JAN 2025', 'DD Month YYYY'), 'companies from china are coming for providing internships on 23 march 2025.')
SELECT * FROM dual;

```

QUERIES

1. This query can track the lecture information such as lecture id, what was the topic of the lecture, who delivered the lecture and the average rating of the lecture by the students.

```

select l.lecture_id,tp.topic_name as topic,
       sp.first_name as delived_by,
       bs.business_name,ROUND(AVG(fdb.rating), 2) AS average_rating,
       TO_CHAR(tmb.start_date, 'DD Month YYYY') as lecture_date FROM lecture l
       JOIN speaker sp USING (speaker_id)
       JOIN topic tp using(topic_id)
       join business bs using(business_id)
       JOIN lecture_feedback fdb on l.lecture_id = fdb.lecture_id
       JOIN time_table tmb on tmb.lecture_id = l.lecture_id
GROUP BY
  l.lecture_id, tp.topic_name, sp.first_name, bs.business_name, tmb.start_date;

```

LECTURE_ID	TOPIC	DELIVED_BY	BUSINESS_NAME	AVERAGE_RATING	LECTURE_DATE
1001	algorithms	aman	ai heath care	5	23 January 2025
1002	relational database	kamla	ai semesters	4	24 January 2025

2. This query will provide all the information about placements.

SELECT

```
st.student_id,  
st.first_name || ' ' || st.last_name AS student_name,  
b.business_name,  
p.placement_type,  
p.status AS placement_status,  
pr.preferred_date,  
sf.first_name || ' ' || sf.last_name AS Arranged_by,  
pf.rating AS feedback_rating,  
pf.comments AS feedback_comments
```

FROM

```
STUDENT st
```

```
JOIN PLACEMENT p ON st.student_id = p.student_id
```

```
JOIN BUSINESS b ON p.business_id = b.business_id
```

```
JOIN STAFF sf ON p.staff_id = sf.staff_id
```

```
JOIN PLACEMENT_REQUEST pr ON p.placement_request_id = pr.request_id
```

```
LEFT JOIN PLACEMENT_FEEDBACK pf ON pf.placement_id = p.placement_id
```

```
ORDER BY st.student_id;
```

RESULT

STUDENT_ID	STUDENT_NAME	BUSINESS_NAME	PLACEMENT_TYPE	PLACEMENT_STATUS	PREFERRED_DATE	ARRANGED_BY	FEEDBACK_RATING	FEEDBACK_COMMENTS
100001	rishab khanna	ai heath care	internship	placed	4/23/2025	anuradha sethi	5	fantastic experience
100002	sanam rana	ai semesters	intership	placed	5/23/2025	rama kanna	5	good

3. This Query will provide important information about speakers like name, their topic expertise, which topic belongs to which module

SELECT

```
s.speaker_id,  
s.first_name,  
s.last_name,  
t.topic_name,  
m.module_name
```

FROM

```
SPEAKER s
```

```
JOIN SPEAKER_EXPERTISE se ON s.speaker_id = se.speaker_id
```

```
JOIN TOPIC t ON se.topic_id = t.topic_id
```

```
JOIN MODULE m ON t.module_id = m.module_id;
```

SPEAKER_ID	FIRST_NAME	LAST_NAME	TOPIC_NAME	MODULE_NAME
2001	aman	khurana	algorithms	machine learning
2001	aman	khurana	relational database	Information systems and databases

4. This will provide the timetable information about a particular lecture, such as date, time, building, room no and about speaker as well.

```
SELECT
    l.lecture_id,
    s.first_name || ' ' || s.last_name AS speaker_name,
    t.topic_name,
    m.module_name,
    tt.floor_building,
    tt.room_no,
    tt.start_date,
    TO_CHAR(tt.start_time, 'HH24:MI') AS start_time
FROM
    LECTURE l
JOIN SPEAKER s ON l.speaker_id = s.speaker_id
JOIN TOPIC t ON l.topic_id = t.topic_id
JOIN MODULE m ON t.module_id = m.module_id
JOIN TIME_TABLE tt ON l.lecture_id = tt.lecture_id;
```

LECTURE_ID	SPEAKER_NAME	TOPIC_NAME	MODULE_NAME	FLOOR_BUILDING	ROOM_NO	START_DATE	START_TIME
1001	aman khurana	algorithms	machine learning	3 pigeon	101	1/23/2025	09:30
1002	kamla sethi	relational database	Information systems and databases	3 pigeon	102	1/24/2025	09:30

5. It will provide the lecture information which has the rating less than three.

```
SELECT
    l.lecture_id,
    t.topic_name,
    sf.rating,
    sf.lecture_feedback
FROM
    LECTURE_FEEDBACK sf
JOIN LECTURE l ON sf.lecture_id = l.lecture_id
JOIN TOPIC t ON l.topic_id = t.topic_id
WHERE sf.rating < 3;
```

LECTURE_ID	TOPIC_NAME	RATING	LECTURE_FEEDBACK
1003	algorithms	2	good

6. This query will provide information about the speaker who did not deliver a single lecture.

```

SELECT
|   s.speaker_id,
|   s.first_name,
|   s.last_name
FROM
|   SPEAKER s
LEFT JOIN LECTURE l ON s.speaker_id = l.speaker_id
WHERE l.lecture_id IS NULL;

```

SPEAKER_ID	FIRST_NAME	LAST_NAME
2003	kamal	malvi

7. This query will help to know how many students were placed by different businesses.

```

SELECT
|   b.business_name,
|   COUNT(DISTINCT p.student_id) AS students_placed
FROM
|   BUSINESS b
JOIN PLACEMENT p ON b.business_id = p.business_id
GROUP BY
|   b.business_name;

```

BUSINESS_NAME	STUDENTS_PLACED
ai semesters	1
ai heath care	1

8. It can help to track the information about the staff who are not active such as don't provide announcements.


```

SELECT
    sf.staff_id,
    sf.first_name,
    sf.last_name
FROM
    STAFF sf
LEFT JOIN ANNOUNCEMENT a ON sf.staff_id = a.posted_by
WHERE a.announcement_id IS NULL;

```

STAFF_ID	FIRST_NAME	LAST_NAME
102	rama	kanna

- Track the staff information which are not connected with businesses.

```

SELECT
    st.staff_id,
    st.first_name,
    st.last_name,
    st.designation
FROM
    STAFF st
LEFT JOIN BUSINESS b
    ON st.staff_id = b.designated_staff_contact_id
WHERE b.business_id IS NULL;

```

STAFF_ID	FIRST_NAME	LAST_NAME	DESIGNATION
105	pooja	singh	professor
104	sam	saini	professor

DATABASE SECURITY

Introduction:

now a days SQL is being used in multifarious of organization to store and retrieve data from the database. Database security is really very indispensable from unauthorised access. Oracle apex provides multiple layers of protection to secure the data from unauthorised users.

There are different ways to protect data such as:

1. Authorisation

Authorization is very indispensable principle to secure data in database. It is all about who can excess the information in database and also what kind of specific operations done by the user is called authorization. It could be very essential in terms of data security as only authorised users can make changes or retrieve information from the database.

in a cooperate HR system, employees can see only their personal details like their appointments, working hours, salary and so on. But HR staff can update salary information, staff details, and so only but the manager can see only performance of their staff but cannot change payment data like salary. So, this shows how crucial is authorization. Specific users only can perform specific tasks only which are necessary for then. Hence because of authorization there is security for sensitive data.

2. Authentication

Authentication is also important in terms of security as it verifies the users by password (unique identifier), and username. it can make sure only verified users can login and perform the tasks with the database and it prevents unauthorised access. Let's take a real time example university database.

Let's assume there is a university, the professors modify the students' grades. There is a username and a password for professors to accomplish the task. If an unauthorised user will try to login with wrong password and user name, the access will be defined.

In a bank when user has to

3. Access

The third security measure could be granting and revoking of privileges. A privilege is the permission to retrieve or modify data from the database. Giving privilege is very important who should have appropriate privilege otherwise database can be compromised if it is not given properly. privileges should be given to those who cannot manage without them or cannot execute their work. The following is the real example that shows how important it is.

Let's assume there is a hospital. Doctors have access to the patients' health status and medical history and a receptionist has access of patient appointments details only. If privileges are not set properly, the receptionist could access the sensitive data which is not secured and big blunder.

4. Encryption

‘If a database system sensitive data, it may be hacked by unauthorised users the chances are high. Some DBMSs provide an encryption facility for this purpose in order to protect data. The DBMS can access the data (after decoding it). Encryption also protects data transmitted over communication lines. There are ample of methods for encoding data to conceal the information’.

5. Integrity

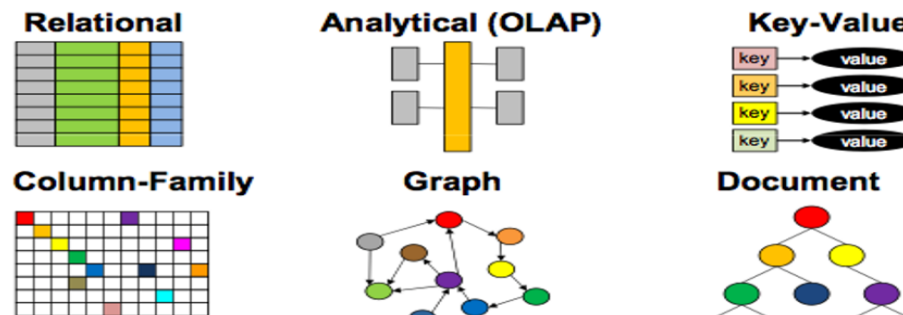
Integrity is must to maintain security in RDBMS, it ensures that the data is trustworthy, consistent. We should add proper constraints like primary key, foreign keys, CHECK constraint on oracle database to make sure the data in database is accurate and arranged in proper manner. Without integrity, data could be corrupted.

For real example let’s assume there are two tables, customers and orders. A customer can order many orders but one order belongs to only one person. There is one to many relationships. If customer id is not put properly in order table as a foreign key then it could cause problems table will not be connected and there will not be results if user tries to retrieve data using joins.

ANOTHER DATABASE

NO SQL databases store data without a structured mechanism to link data from different tables to one another. The indispensable advantage is that it needs low – cost hardware as compared to relational databases. It is faster than relational databases in operations such as (read/write) and best for online analytical processing. It has no fixed columns as relational database hence it is free from schema. ACID transaction properties are not required. Some examples of no SQL databases such as MongoDB, Redis, Cassandre and so on. There are different types of databases like column, key-value, and document.

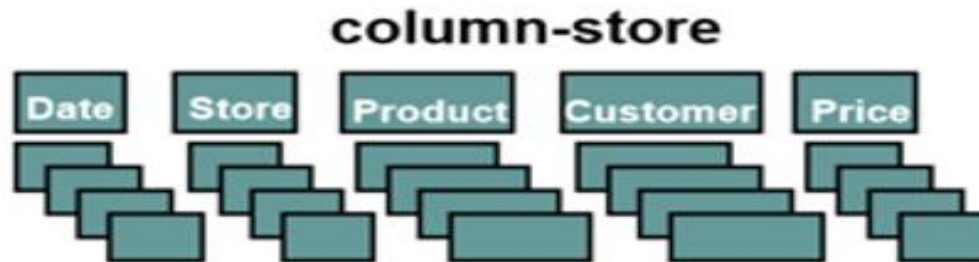
NoSQL Database Patterns



COLUMNAR DATABASE

It is quite different from relational database because it uses columns to store data instead of rows to store data. It is memory efficient; it takes less space in memory as compared to relational database. It is used to store and manipulate data large data effectively. Arithmetic operations like min, max average is quite easy to calculate as compared to RDBMS. 'Columnar databases store all the cells relating to a column as a continuous disk entry faster search/accesses'.

Example of columnar database is Cassandra.



Example:

It is used in e commerce platforms to a great extent for analytical process, such as to find out what is total revenue, what is top selling product, average sales and so on. It is used specifically because it is very easy to do arithmetic operation by using the columnar database such as average, addition, multiplication and many more.

DOCUMENT DATABASE

Document databases store semi-structured data like JSON or XML that make it easy for developers to work with the same format used in their code. They offer flexible schemas which is best for dynamic data that may change over time. This type of data base is best for storing catalogs. Apps like Netflix, amazon, amazon prime use this database. This database is best for apps or websites where there is constant change in data of users in a very large amount and where there is need to make catalog for example recommendation systems on Netflix and products listing on shopping websites.

Example:

On amazon when a user searches a query like shampoo, all the products are shown which are categorized as shampoo. In a document database, each product is stored as a self-contained document (usually in JSON format), containing fields like name, category, brand, price, tags this helps to query based on specific such as 'shampoo'.

```
json

{
  "product_id": "101",
  "name": "Herbal Shampoo",
  "category": "Shampoo",
  "brand": "HerbalCare",
  "price": 5.99,
  "tags": ["herbal", "natural", "shampoo"],
  "ingredients": ["aloe vera", "coconut oil"],
  "hair_type": "Dry",
  "reviews": 4.5
}
```

KEY – VALUE DATABASE

A key – value DB is a simple DB that uses an associative as a fundamental model. Where each key is connected with one and only one value in a collection. It is very simple database. Operations on it for query are fast on it and it allows horizontal scaling. One of the most powerful uses of this database is caching, it stores frequently accessed data temporarily in RAM instead of going to the main database again and again every time. It not only saves the resources of the organization as well improve the speed of the app or website.

Example : Amazon ElastiCache for Product Page Loading:

If multiple users are going to same product, Amazon uses ElastiCache (a key-value store) to store the data temporarily in order to give fast results, save resources, and reducing burden on the database.

GRAPH DATABASE

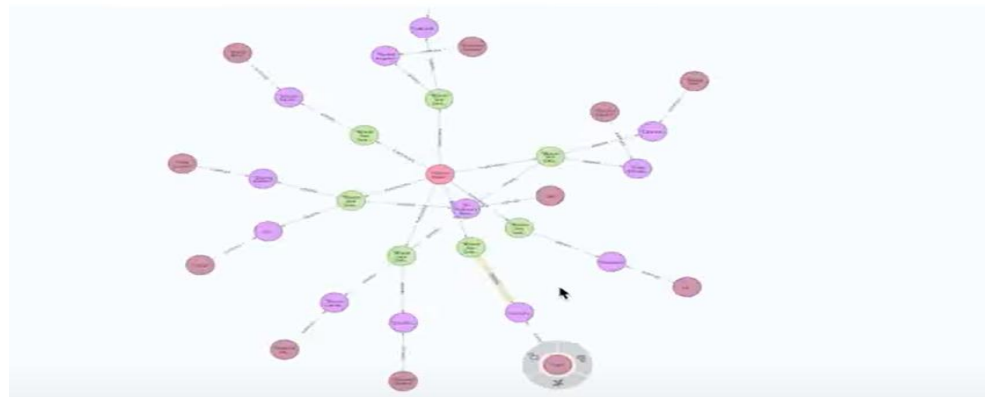
A graph DB is basically a collection a collection of nodes and edges. ‘Each node represents an entity and each edge represent a connection between two nodes’. It is very useful for large data’, in this database’ there is no need to do computation on query time data is just there after being computed, node relationships are

already pre-computed' just user has to traverse only for collecting information. More deep data can be retrieved from the database as compared to RDBMS. Let's assume a scenario:

In social networking apps like Facebook a user has different activities like doing comments, like pictures, following and so on. It has billions of users so different users execute these activities hence to track on these activities like who is whose friend, who follow who, who's like who' picture the Graph Database is beneficial for such kind of social apps. Inclusion, for networking apps or website this database is beneficial and efficient.

Example - Neo4j Aura DB

Applications: representing social networks, pattern detection etc



REFERENCES:

<https://www.mongodb.com/resources/basics/databases/document-databases>

<https://www.youtube.com/watch?v=H-A7VdONggM>

lecture notes

<https://www.youtube.com/watch?v=NvjKXwt7n48>

Database systems: a practical approach to design, implementation and management by Connolly Thomas

